



清华大学

Tsinghua University

前排签到

下载并上传 `ibex_work_upload.zip` 到服务器





清华大学

Tsinghua University

From RTL to GDS 设计流程讲解 (中)

霍德萱, 张吉霖, 陈虹

THU-SIC

2023年5月



目录



清华大学
Tsinghua University

- 综合
- 形式验证
- 布局
- 布线



布局 and 布线 (一)



- PR data init: 输出PR需要的必要文件

输入	输出
ibex_core.syn.v sky130_fd_sc_hd_merged.lef sky130_fd_sc_hd.tlef sky130_fd_sc_hd_tt_025C_1v80.lib	init_design.enc init_design.enc.dat init.cmd init.log init.logv init_data_timing check_data_init.report

名称	说明
ibex_core.syn.v	DC综合后的网表
sky130_fd_sc_hd_merged.lef	描述工艺库的物理特性的文件 (尺寸、引脚位置、电容、电感)
sky130_fd_sc_hd.tlef	
sky130...1v80.lib	描述工艺库的电气特性的文件 (时序、功耗、电压)

布局和布线（一）



- PR data init

名称	说明
init_design.enc	芯片设计的初始状态信息， 如逻辑元素的布局、连接、时序
init_design.enc.dat	
init.cmd	PR工具的初始化命令和参数
init.log	记录PR工具的初始化过程和结果
init.logv	
init_data_timing	芯片设计的数据初始化信息，如逻辑 元素的状态、时钟设置
check_data_init.report	芯片设计的时序分析结果，如时钟频 率、时序路径、时序违规等



布局和布线（一）



- PR data init

Innovus为Cadence于2015年发布的数字芯片P&R布局布线版图设计工具，
替代以前的Encounter平台

```
data_init:
    cd $(RESULT_DIR); mkdir -p pr/data; \
        mkdir -p pr/log; \
        mkdir -p pr/report;
    innovus -files $(SCRIPTS_DIR)/pr/init.tcl \
        -log "$(RESULT_DIR)/pr/log/init" -overwrite
```

- 运行命令

运行此步骤前，请确保综合流程已经成功运行，并且通过形式化验证

```
make data_init
```



布局和布线（一）



- PR data init

运行脚本讲解 1 – 配置

```
init:
set defHierChar {/}  设置层次分隔符为 “/”
set init_gnd_net {VSS} 设置初始接地网络为 “VSS”
set init_pwr_net {VDD} 设置初始电源网络为 “VDD”           设置网表文件路径
set init_verilog $::env(RESULT_DIR)/syn/data/$::env(DESIGN_NAME).syn.v
set init_top_cell $::env(DESIGN_NAME) 设置顶层单元名字
set init_lef_file [split $::env(LEF_FILES)] 设置LEF文件路径
puts $::env(SCRIPTS_DIR)/pr/mmmc.view  输出mmmc.view文件的路径
set init_mmmc_file $::env(SCRIPTS_DIR)/pr/mmmc.view 设置mmmc.view路径
init_design 开始设计
```

./scripts/pr/init.tcl

mmmc.view是MMMC（Multi-Million Macrocell Compiler）的一个视图文件，包含了设计的物理信息和约束，用于后续的布局和布线操作。

布局和布线（一）



- PR data init

运行脚本讲解 2 – 检查及生成报告

```
checkDesign -netList -noHtml -outfile  
$::env(RESULT_DIR)/pr/report/check_data_init.report
```

生成检查报告，检查设计是否初始化成功

```
timeDesign -prePlace \ 计算设计程序，并生成相应报告  
-pathReports \ 生成路径报告  
-drvReports \ 生成驱动器报告  
-slackReports \ 生成时序报告  
-numPaths 50 \ 生成的时序路径数量  
-prefix prePlace \ 指定报告文件名的前缀  
-outDir $::env(RESULT_DIR)/pr/report/init_data_timing  
指定输出报告的路径
```

```
saveDesign $::env(RESULT_DIR)/pr/data/init_design.enc: 保存设计  
exit: 退出脚本
```

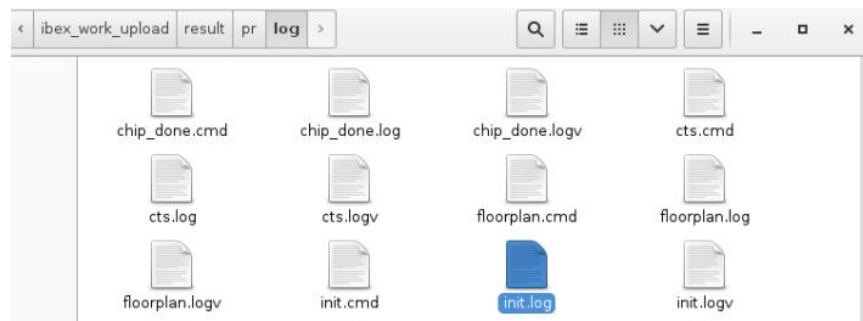
./scripts/pr/init.tcl

布局 and 布线 (一)



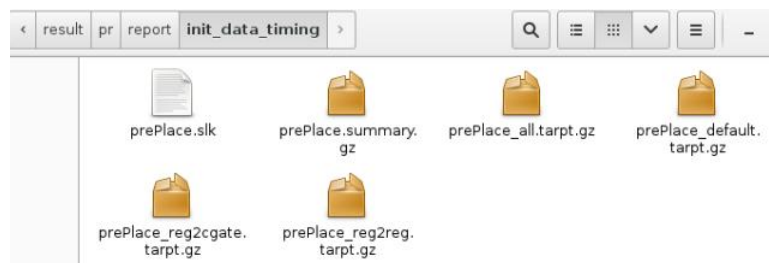
清华大学
Tsinghua University

• PR data init 查看结果

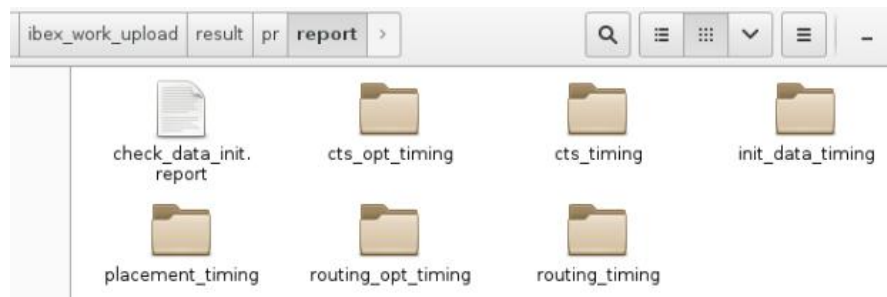


log文件用于记录PR
data init的分析报告

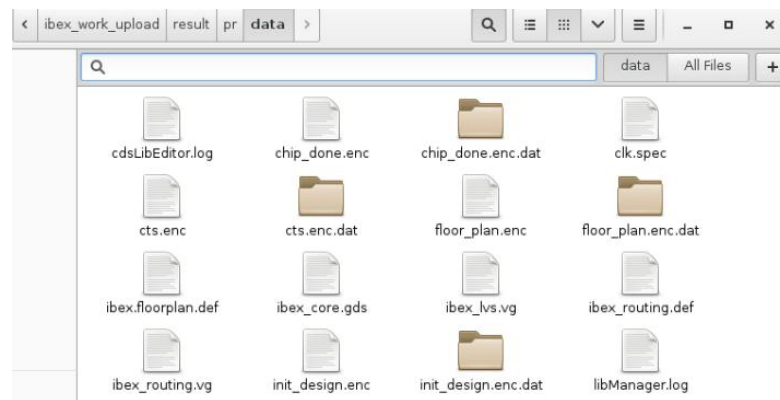
检查有无error



具体的设计结果



检查报告
记录了每一个电路单元的信息



输出的文件路径

布局和布线（二）



- Floor plan:

- 步骤目标

完成芯片面积的设定与io的摆放

- 输入&输出

输入	输出
init_design.enc	floorplan.cmd floorplan.log floorplan.logv floor_plan.enc.dat floor_plan.enc 二进制文件，记录了芯片的物理布局 记录了芯片的物理信息和约束

- 运行命令

运行此步骤前，请确保PR data init已经成功运行

```
make floorplan
```



布局和布线 (二)



- Floor plan:

```
floorplan:
> innovus -files $(SCRIPTS_DIR)/pr/floor_plan.tcl \
> -log "$(RESULT_DIR)/pr/log/floorplan" -overwrite
```

运行脚本讲解 1

```
set begin [clock format [clock seconds] -format %Y%m%d_%l:%M_%p]
puts "The FloorPlan Start: $begin"
source $::env(RESULT_DIR)/pr/data/init_design.enc
floorPlan -site $::env(PLACE_SITE) -su 1 $::env(PLACE_DENSITY) 1 1 1 1
if {[info exists ::env(IO_FILE)]} {
    if {$::env(IO_FILE)!=""} {
        loadIoFile $::env(IO_FILE)
    } else {
        source $::env(SCRIPTS_DIR)/pr/place_io.tcl
    }
} else {
    source $::env(SCRIPTS_DIR)/pr/place_io.tcl
}
```

Floorplan开始的时间

读取xxx.enc文件(物理设计和约束)

环境变量

定义芯片的可布局区域, 并设置布局密度需修改! 具体尺寸后续进行通知。

根据输入的IO文件或者脚本, 放置芯片的输入输出端口。

```
#floorPlan -site $::env(PLACE_SITE) -su 1 $::env(PLACE_DENSITY) 35 35 35 35
floorPlan -site $::env(PLACE_SITE) -d {2400 1300 50 50 50 50}
```

./scripts/pr/floor_plan.tcl

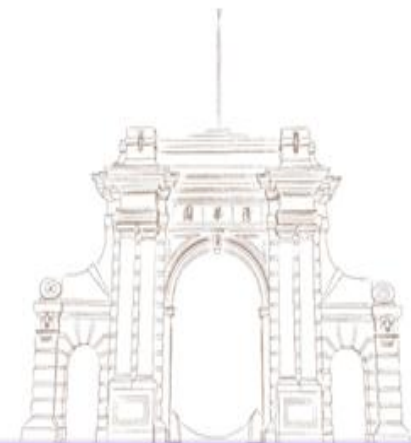
布局 and 布线 (二)



- Floor plan:

修改供电方案

```
3 # -----
4 addInst -cell PCORNER      -inst CornerCellN
5 addInst -cell PCORNER      -inst CornerCellS
6 addInst -cell PCORNER      -inst CornerCellE
7 addInst -cell PCORNER      -inst CornerCellW
8
9 addInst -cell PVDD1CDG      -inst POWER_VDDN
10 addInst -cell PVDD1CDG      -inst POWER_VDDS
11 addInst -cell PVDD1CDG      -inst POWER_VDDE
12 addInst -cell PVDD1CDG      -inst POWER_VDDW
13
14 addInst -cell PVSS1CDG      -inst POWER_VSSN
15 addInst -cell PVSS1CDG      -inst POWER_VSSS
16 addInst -cell PVSS1CDG      -inst POWER_VSSE
17 addInst -cell PVSS1CDG      -inst POWER_VSSW
18
19 addInst -cell PVDD2CDG      -inst POWER_VDDION0
20 addInst -cell PVDD2CDG      -inst POWER_VDDION1
21 addInst -cell PVDD2CDG      -inst POWER_VDDIOS0
22 addInst -cell PVDD2CDG      -inst POWER_VDDIOS1
23 addInst -cell PVDD2CDG      -inst POWER_VDDIOE0
24 addInst -cell PVDD2CDG      -inst POWER_VDDIOE1
25 addInst -cell PVDD2CDG      -inst POWER_VDDIOW0
26 addInst -cell PVDD2POC      -inst PowerOnControl
27
28 addInst -cell PVSS2CDG      -inst POWER_VSSION0
29 addInst -cell PVSS2CDG      -inst POWER_VSSION1
30 addInst -cell PVSS2CDG      -inst POWER_VSSIOS0
31 addInst -cell PVSS2CDG      -inst POWER_VSSIOS1
32 addInst -cell PVSS2CDG      -inst POWER_VSSIOE0
33 addInst -cell PVSS2CDG      -inst POWER_VSSIOE1
34 addInst -cell PVSS2CDG      -inst POWER_VSSIOW0
35 addInst -cell PVSS2CDG      -inst POWER_VSSIOW1
36
```



布局和布线 (二)



- Floor plan:

运行脚本讲解 2

```
foreach cell [split $::env(DONT_USE_CELLS)] {  
    get_lib_cell */$cell  
    set_dont_use [get_lib_cells */$cell] true  
}
```

对于不需要使用的单元，设置其不可使用

```
set endcap_right "right_tap_cell_name"
```

```
set endcap_left "left_tap_cell_name"
```

芯片四个边上的端口单元名称进行命名

```
set endcap_top "left_tap_cell_name"
```

```
set endcap_bottom "left_tap_cell_name"
```

```
setEndCapMode -reset 重置
```

**设置芯片四个边上的端口单元名称，
端口名字前缀为ENDCAP**

```
setEndCapMode -leftEdge $endcap_left -rightEdge $endcap_right -topEdge
```

```
$endcap_top -bottomEdge $endcap_bottom -prefix ENDCAP
```

./scripts/pr/floor_plan.tcl

布局 and 布线 (二)



- Floor plan:

运行脚本讲解 3

```
saveDesign $::env(RESULT_DIR)/pr/data/floor_plan.enc 保存Floorplan的enc文件
defOut -floorplan -noStdCells $::env(RESULT_DIR)/pr/data/ibex.floorplan.def
输出Floorplan生成的def文件
set end [clock format [clock seconds] -format %Y%m%d_%l:%M_%p]puts "The
FloorPlan End: $end" Floorplan结束时间
Exit
```

./scripts/pr/floor_plan.tcl



布局和布线 (二)



- Floor plan: 报告讲解

用less命令可以打开.def文件

实际用到的区域往往会比规划的区域要少一点

```
#####  
# Generated by: Cadence Innovus 19.15-s075_1  
# OS: Linux x86_64(Host ID edaserver)  
# Generated on: Sun May 7 23:40:56 2023  
# Design: ibex_core  
# Command: defOut -floorplan -noStdCells ./result/pr/data/ibex.floorplan.def  
#####
```

```
VERSION 5.8 ;  
DIVIDERCHAR "/" ;  
BUSBITCHARS "[]" ;  
DESIGN ibex_core ;  
UNITS DISTANCE MICRONS 1000 ; 这是物理布局单位的定义，这里是以微米为单位
```

```
PROPERTYDEFINITIONS  
  DESIGN flow_implementation_stage STRING "place_opt" ;  
  COMPONENTPIN designRuleWidth REAL ;  
  DESIGN FE_CORE_BOX_LL_X REAL 1.3800 ;  
  DESIGN FE_CORE_BOX_UR_X REAL 554.7600 ; 实际用到的区域  
  DESIGN FE_CORE_BOX_LL_Y REAL 1.0200 ;  
  DESIGN FE_CORE_BOX_UR_Y REAL 553.1800 ;  
END PROPERTYDEFINITIONS
```

```
DIEAREA ( 0 0 ) ( 556140 554200 ) ; 芯片的物理边界，用于规定芯片的大小和位置
```

```
ROW CORE_ROW_0 unithd 1380 1020 FS DO 1203 BY 1 STEP 460 0  
;  
ROW CORE_ROW_1 unithd 1380 3740 N DO 1203 BY 1 STEP 460 0  
;  
ROW CORE_ROW_2 unithd 1380 6460 FS DO 1203 BY 1 STEP 460 0  
;  
ROW CORE_ROW_3 unithd 1380 9180 N DO 1203 BY 1 STEP 460 0  
;  
ROW CORE_ROW_4 unithd 1380 11900 FS DO 1203 BY 1 STEP 460 0  
;  
ROW CORE_ROW_5 unithd 1380 14620 N DO 1203 BY 1 STEP 460 0  
;
```

DO 1203 BY 1表示该行中有1203个单元格，在垂直方向上只有一行

STEP 460 0表示相邻单元格的间距为460个单位高度

起始位置 排列方向: FS从左到右; N从下到上



布局和布线 (三)



- Power plan:

- 步骤目标

完成对设计的电源网络规划,并完成connectivity检查

- 输入&输出

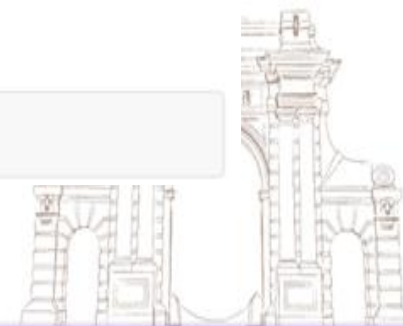
输入	输出
floor_plan.enc	powerplan.enc powerplan.enc.dat power_plan.cmd power_plan.log power_plan.logv

后端中生成的文件格式大体相同, 其中enc文件最为重要, 是用于进行下一步的关键文件

- 运行命令

执行此步骤前, 请确保Floorplan步骤成功执行,并输出了floorplan.enc

```
make power_plan
```



布局和布线 (三)



- Power plan:

```
power_plan:
> innovus -files $(SCRIPTS_DIR)/pr/power_plan.tcl \
> -log "$(RESULT_DIR)/pr/log/power_plan" -overwrite
```

运行脚本讲解 1

`source $::env(RESULT_DIR)/pr/data/floor_plan.enc` 导入floorplan.enc文件

Global PG net connect

将VDD全局电源连到所有单元的pgpin和tiehi类型引脚上

`globalNetConnect VDD -type pgpin -pin "$::env(PWR_PORT)" -inst *`

`globalNetConnect VDD -type tiehi -pin "$::env(PWR_PORT)" -inst *`

`globalNetConnect VDD -type net -net VDD` 将VDD全局电源引脚连接到VDD电源网络

`globalNetConnect VSS -type pgpin -pin "$::env(GND_PORT)" -inst *`

`globalNetConnect VSS -type tielo -pin "$::env(GND_PORT)" -inst *`

`globalNetConnect VSS -type net -net VSS`

VSS同上

`./scripts/pr/power_plan.tcl`

布局 and 布线 (三)



- Power plan:

类型	引脚	解释
pgpin	电源-地	pgpin被用于连接电源和地面引脚。它们通常是在芯片的周边区域放置的，以便方便地连接到电源和地面网络。
tiehi	电源-芯片内部	tiehi被用于将电源引脚连接到芯片内部的电源网络。它通常被用于芯片的电源引脚，以便将电源引脚连接到芯片内部的电源网络。 tiehi引脚的作用是提供一个低阻抗的路径，以确保电源信号能够在芯片内部快速传输。



布局和布线 (三)



- Power plan:

运行脚本讲解 2

```
# -----  
# Add the power stripe  
# -----  
addStripe -nets {VSS VDD} \ 增加电源VSS, VDD 指定电源条的金属层,  
    -layer $::env(V_STRIPE_METAL) \ 使用环境变量V_STRIPE_METAL  
    -direction vertical \ 指定电源条的方向, 这里是垂直方向  
    -width $::env(STRIPE_WIDTH) \ 指定电源条的宽度(环境变量STRIPE_WIDTH)  
    -spacing $::env(STRIPE_SPACING) \ 指定电源条的间距(环境变量STRIPE_SPACING)  
    -set_to_set_distance $::env(STRIPE_DISTANCE) \  
    -start_from left \ 指定电源条之间的距离(环境变量STRIPE_DISTANCE)  
    -start_offset 1 \  
    -uda power_stripe_vadd
```

指定电源条起始位置和起始偏移量,
用户定义属性uda(如果有特定需求的走线需要自行添加)

./scripts/pr/power_plan.tcl

布局和布线（三）

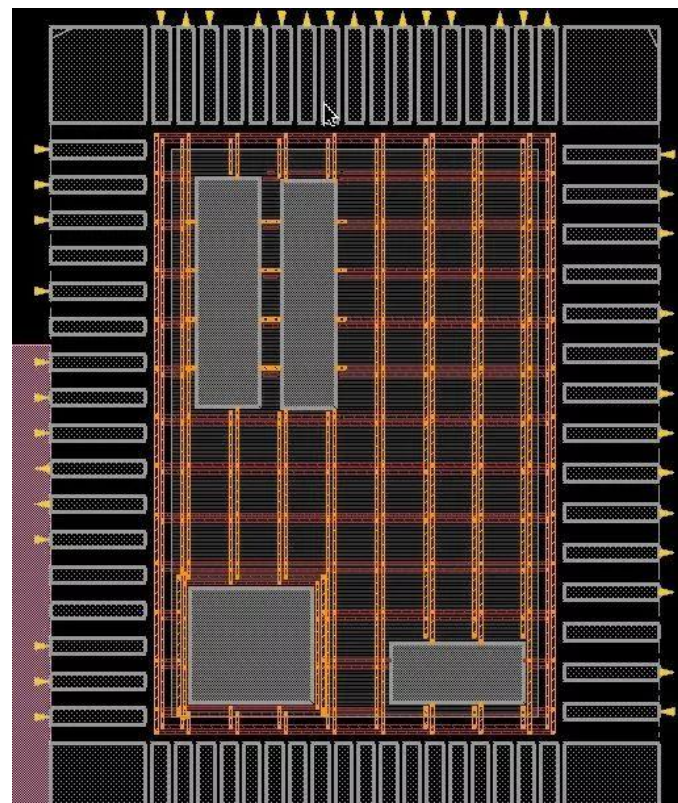


- Power plan:

Power Stripe。中文名就是电源条线。我们把芯片core里面纵横交错的电源网格称为Power stripe。

它的作用就是给design中的hard macro以及power rail供电，将电源信息传递给core里面各个角落。

通常按照一定的pattern规律添加，基本都会选用高层金属走线。因为高层金属具有较小的电阻，可以有效的降低IR drop的影响。对于power stripe宽度和间距设计。通常受到不同工艺的影响，区别会比较大，



布局 and 布线 (三)



- Power plan:

运行脚本讲解 3

```
# -----  
# Add the power stripe  
# -----  
addStripe -nets {VSS VDD} \  
    -layer $::env(V_STRIPE_METAL) \  
    -direction horizontal \  
    -width $::env(STRIPE_WIDTH) \  
    -spacing $::env(STRIPE_SPACING) \  
    -set_to_set_distance $::env(STRIPE_DISTANCE) \  
    -start_from left \  
    -start_offset 1 \  
    -uda power_stripe_h
```

基本同上~

[./scripts/pr/power_plan.tcl](#)

布局 and 布线 (三)



- Power plan:

运行脚本讲解 4

```
# -----  
# Add the power rail 添加电源轨  
# -----  
set srout_min_layer $::env(SROUTE_MIN_LAYER) 指定电源轨布线时使用的最小  
set srout_max_layer $::env(SROUTE_MAX_LAYER) 和最大金属层数量  
srout -connect { corePin } \ 执行电源轨的布线, 将其连接到核心引脚 (corePin) 上  
    -layerChangeRange " $srout_min_layer $srout_max_layer " \ 指定电源轨的  
    -corePinTarget { none } \ 表示在布线时不考虑核心引脚的位置 金属层范围  
    -allowJogging 1 \ 允许电源轨在布线时出现拐点, 以适应布线的需要 指定电源轨  
    -crossoverViaLayerRange " $srout_min_layer $srout_max_layer " \ 的穿越金属  
    -nets { VDD VSS } \ 指定连接的电源网络, 即VDD和VSS 层范围  
    -allowLayerChange 1 \ 允许电源轨在布线时切换金属层, 以适应布线的需要  
    -targetViaLayerRange " $srout_min_layer $srout_max_layer " \  
    -uda power_rail 指定电源轨的目标金属层范围
```

./scripts/pr/power_plan.tcl

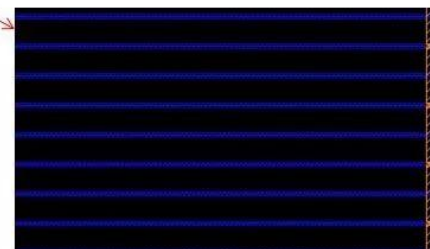
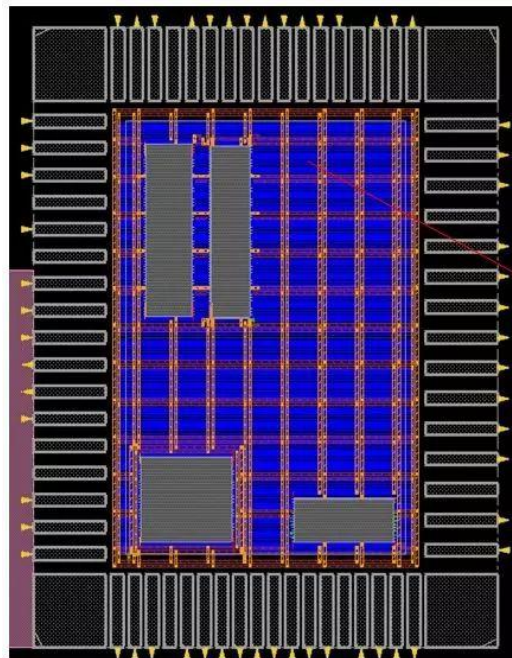
布局 and 布线 (三)



- Power plan:

中文名电源轨道，这是我们在power plan中经常用到的一个概念。

power rail主要是形成standard cell的供电网络，并将其与芯片core里面的电源网络相连，比如说电源环线power ring和电源条线power stripe。如右图所示（蓝色部分）：



每条power rail都需要加在core里面的row上，由于standard cell的电源pin都在顶和底上面。这样当standard cell做完placement，摆放到row上以后，就能自动地接上power rail上的供电，有点像火车轨道一样



布局和布线 (三)



- Power plan:

运行脚本讲解 5

```
# -----  
# Verify connect violation 检查连接性  
# -----  
verifyConnectivity -type special \ 指定检查类型为特殊连接，即检查特殊连接的正确性  
执行连接性检查 -noAntenna \ ban天线效应检查(金属线和电源网络)  
-noWeakConnect \ ban检查那些连接信号弱的网络  
-noUnroutedNet \ ban检查那些没有被正确连接的网络  
-error 1000 \ 如果出现了1000个或更多的错误，则认为检查失败  
-warning 50 如果出现了50个或更多的警告，则认为检查有问题  
  
verify_PG_short  
检查电源引脚短路 -no_routing_blk  
ban检查芯片内部不同块之间的连接
```

./scripts/pr/power_plan.tcl

布局和布线（三）

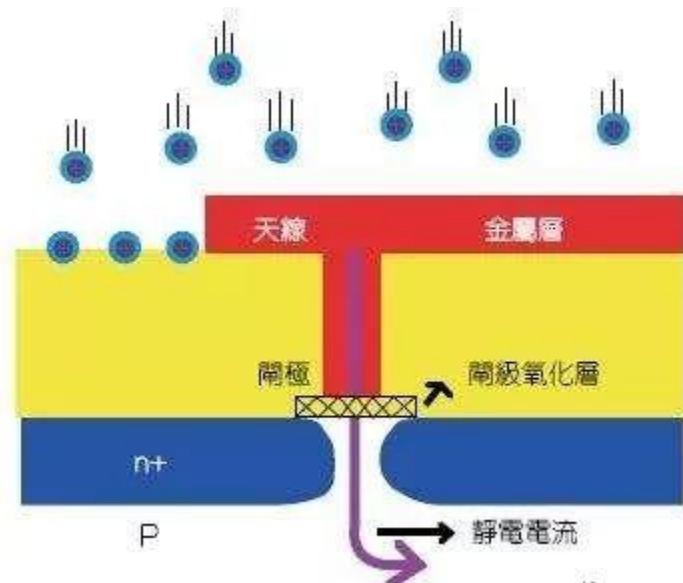


- Power plan:

天线效应 (antenna effect) 是指金属线和电源网络之间的相互作用，通常会导致信号干扰和电源噪声等问题。

在芯片生产过程中，暴露的金属线或者多晶硅(polysilicon)等导体，就象是一根根天线，会收集电荷（如等离子刻蚀产生的带电粒子）导致电位升高。

天线越长，收集的电荷也就越多，电压就越高。若这片导体碰巧只接了MOS 的栅，那么高电压就可能把薄栅氧化层击穿，使电路失效，这种现象我们称之为“天线效应”。



布局 and 布线 (三)

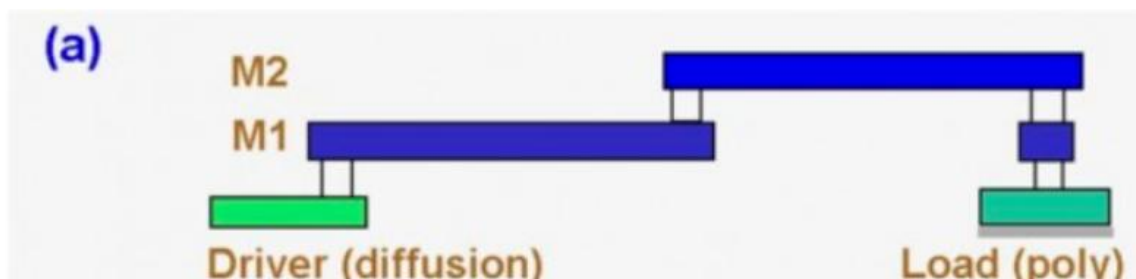


- Power plan:

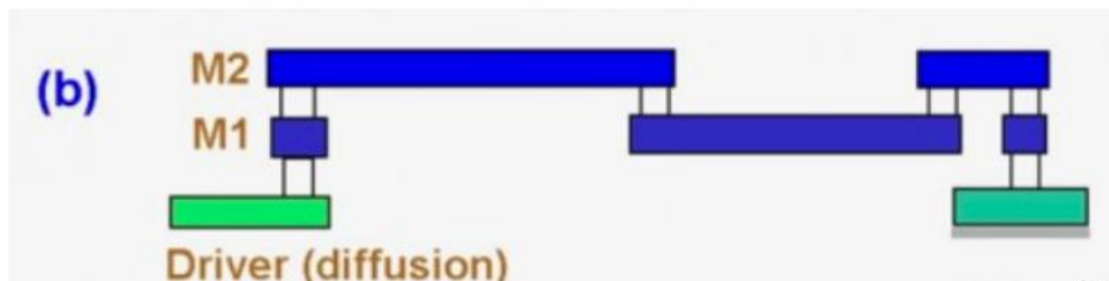
天线效应解决方法:

(1) 高层走线: M1 层走信号, 为避免长走线, 先跳到 M2 或高层金属, 再直接连接栅极。

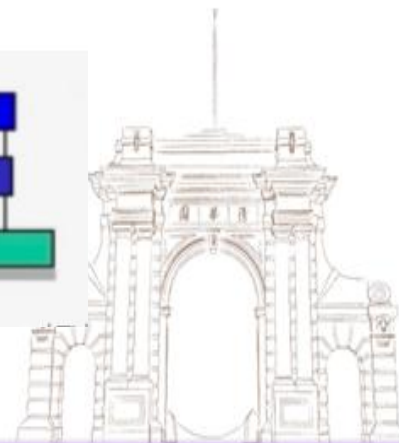
注意: 连接栅极的高层金属面积也不能过大



(2) 向上跳线: 和第一种方法类似, M1 层走信号, 向上跳线到 M2 层再连接栅极。



还有保护二极管法等等。核心思想就是将MOS的G端积累的电荷泄放掉



布局和布线 (三)



- Power plan:

运行脚本讲解 6

```
# -----  
# Save design  
# -----  
saveDesign $::env(RESULT_DIR)/pr/data/powerplan.enc  
exit
```

生成powerplan.enc文件，
下一步要用！

./scripts/pr/power_plan.tcl

enc文件是加密文件的扩展名，通常是指经过加密处理的文件。加密是一种安全措施，可以将文件中的数据转换为一种难以理解和破解的形式，从而保护文件的机密性和完整性。

布局和布线 (三)



- Power plan:

报告讲解 1

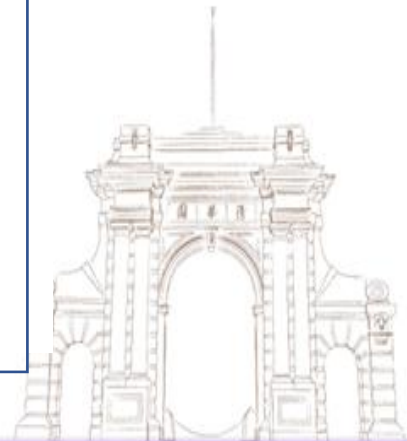
Summary of Active RC-Corners :

Analysis View: max_view

RC-Corner Name	: rc_max
RC-Corner Index	: 0
RC-Corner Temperature	: 25 Celsius
RC-Corner Cap Table	: ''
RC-Corner PreRoute Res Factor	: 1.05
RC-Corner PreRoute Cap Factor	: 1.05
RC-Corner PostRoute Res Factor	: 1.05 {1.05 1 1}
RC-Corner PostRoute Cap Factor	: 1.05 {1.05 1 1}
RC-Corner PostRoute XCap Factor	: 1.05 {1.05 1 1}
RC-Corner PreRoute Clock Res Factor	: 1.05
RC-Corner PreRoute Clock Cap Factor	: 1.05 [Derived from Cap Table]
RC-Corner PostRoute Clock Cap Factor	: 1.05 {1.05 1 1}
RC-Corner PostRoute Clock Res Factor	: 1.05 {1.05 1 1}

Analysis View: min_view

RC-Corner Name	: rc_min
RC-Corner Index	: 1
RC-Corner Temperature	: 25 Celsius
RC-Corner Cap Table	: ''
RC-Corner PreRoute Res Factor	: 1
RC-Corner PreRoute Cap Factor	: 1
RC-Corner PostRoute Res Factor	: 1 {1 1 1}
RC-Corner PostRoute Cap Factor	: 1 {1 1 1}
RC-Corner PostRoute XCap Factor	: 1 {1 1 1}
RC-Corner PreRoute Clock Res Factor	: 1
RC-Corner PreRoute Clock Cap Factor	: 1 [Derived from Cap Table]
RC-Corner PostRoute Clock Cap Factor	: 1 {1 1 1}
RC-Corner PostRoute Clock Res Factor	: 1 {1 1 1}



布局 and 布线 (三)



• Power plan: 报告讲解 2

```
#% End sroute (date=05/07 23:41:55, total cpu=0:00:00.5, real=0:00:00.0, peak res=846.2M, current mem=836.6M)
<CMD> verifyConnectivity -type special -noAntenna -noWeakConnect -noUnroutedNet -error 1000 -warning 50
VERIFY_CONNECTIVITY use new engine.
```

```
***** Start: VERIFY CONNECTIVITY *****
Start Time: Sun May 7 23:41:55 2023
```

```
Design Name: ibex_core
Database Units: 1000
Design Boundary: (0.0000, 0.0000) (556.1400, 554.2000)
Error Limit = 1000; Warning Limit = 50
Check specified nets
```

```
Net boot_addr_i[0]: Found a geometry with bounding box (0.00,-0.00) (0.14,0.49) outside the design boundary.
Violations for such geometries will be reported.
```

```
**WARN: (IMPVFC-97): I0 pin test_si of net test_si has not been assigned. Please make sure it is assigned and rerun verifyConnectivity.
```

```
**WARN: (IMPVFC-97): I0 pin test_so of net test_so has not been assigned. Please make sure it is assigned and rerun verifyConnectivity.
```

```
**** 23:41:55 **** Processed 5000 nets.
```

```
**** 23:41:55 **** Processed 10000 nets.
```

Begin Summary

Found no problems or warnings.]

End Summary

```
End Time: Sun May 7 23:41:56 2023
Time Elapsed: 0:00:01.0
```

```
***** End: VERIFY CONNECTIVITY *****
Verification Complete : 0 Viols. 0 Wrngs.
(CPU Time: 0:00:00.1 MEM: 0.000M)
```

连接性检查!

```
<CMD> verify_PG_short -no_routing_blk
*** Starting VERIFY PG SHORT(MEM: 921.6) ***
```

```
VERIFY PG SHORT ..... Initializing
VERIFY PG SHORT ..... Deleting Existing Violations
VERIFY PG SHORT ..... Creating Sub-Areas
..... bin size: 2240
VERIFY PG SHORT ..... SubArea : 1 of 4
VERIFY PG SHORT ..... Short: 0 Viols.
VERIFY PG SHORT ..... SubArea : 2 of 4
VERIFY PG SHORT ..... Short: 0 Viols.
VERIFY PG SHORT ..... SubArea : 3 of 4
VERIFY PG SHORT ..... Short: 0 Viols.
VERIFY PG SHORT ..... SubArea : 4 of 4
VERIFY PG SHORT ..... Short: 0 Viols.
Verification Complete : 0 Short Viols.
```

布局和布线 (三)



- Power plan: 报告讲解 3

```
<CMD> verify_PG_short -no_routing_blk  
*** Starting VERIFY PG SHORT(MEM: 921.6) ***
```

```
VERIFY PG SHORT ..... Initializing  
VERIFY PG SHORT ..... Deleting Existing Violations  
VERIFY PG SHORT ..... Creating Sub-Areas  
                        ..... bin size: 2240  
VERIFY PG SHORT ..... SubArea : 1 of 4  
VERIFY PG SHORT ..... Short: 0 Viols.  
VERIFY PG SHORT ..... SubArea : 2 of 4  
VERIFY PG SHORT ..... Short: 0 Viols.  
VERIFY PG SHORT ..... SubArea : 3 of 4  
VERIFY PG SHORT ..... Short: 0 Viols.  
VERIFY PG SHORT ..... SubArea : 4 of 4  
VERIFY PG SHORT ..... Short: 0 Viols.  
Verification Complete : 0 Short Viols.
```

```
*****End: VERIFY PG SHORT*****
```

```
*** verify PG short (CPU: 0:00:00.2 MEM: 75.8M)
```

检查电源引脚短路!



布局 and 布线 (四)



- Placement:

placement是指将逻辑电路元件（如门、寄存器等）放置在芯片的物理位置上的过程

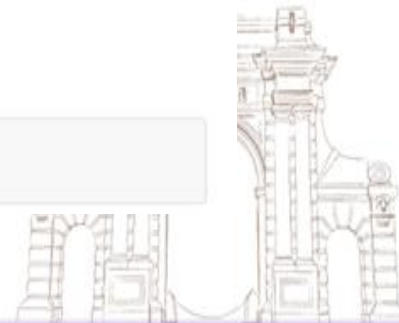
- 输入&输出

输入	输出
floor_plan.enc	powerplan.enc powerplan.enc.dat power_plan.cmd power_plan.log power_plan.logv placement_timing

- 运行命令

运行此步骤前，确保Powerplan步骤成功执行

```
make placement
```



布局和布线 (四)



- Placement:

运行脚本讲解 1

导入powerplan.enc文件，它用于在
placement过程中指定电源和地线的位置

```
source $::env(RESULT_DIR)/pr/data/powerplan.enc
defln $::env(RESULT_DIR)/scanchain/data/ibex_core.scan.def
    导入ibex_core.scan.def文件，它用于定义扫描链的信息
```

```
# -----
# Timing Derate setting
# -----
```

```
set_timing_derate -delay_corner {delay_max} -early 0.97 -late 1.03 -clock
set_timing_derate -delay_corner {delay_max} -late 1.05 -data
setAnalysisMode -cppr both
```

针对数据路径滞后时间延迟减少5%

设置分析模式为both，意味着同时进行时
序路径分析和电源噪声分析

./scripts/pr/placement.tcl

设置时序延迟。-early 0.97表示在时序分析中，将提前时间的延迟减少3%。-late 1.03表示在时序分析中，将滞后时间的延迟减少3%。-clock表示该设置针对时钟路径。

布局和布线 (四)



- Placement:

运行脚本讲解 2

```
# -----  
# Path group setting  
# -----  
reset_path_group -all 重置所有的路径信息 定义寄存器变量, 包括非时钟门控的寄存器  
set reg [filter_collection [all_registers] "is_integrated_clock_gating_cell != true"]  
set input [all_inputs]  
set output [all_outputs] 定义输入/输出变量, 包括所有的输入/输出信号  
set ckgating [filter_collection [all_registers] "is_integrated_clock_gating_cell == true"]  
set ignore_path_groups [list inp2reg reg2out reg2out feedthr]  
定义一个忽略路径组的变量, 其中包含inp2reg、reg2out、feedthr等路径组。  
定义寄存器变量, 包括时钟门控的寄存器
```

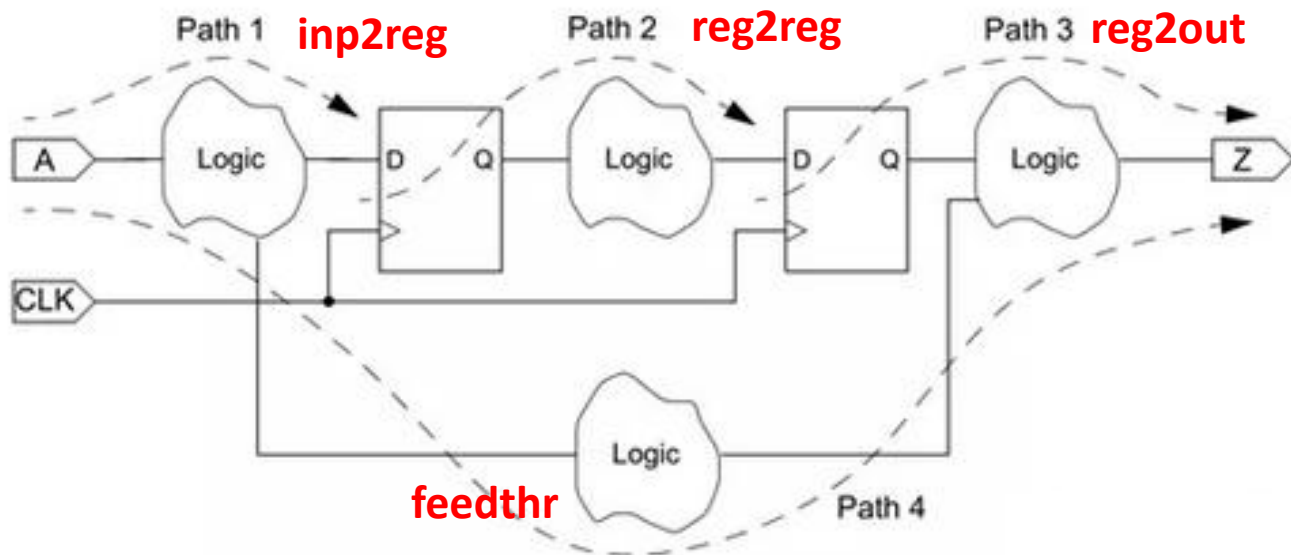
./scripts/pr/placement.tcl

布局和布线（四）



- Placement:

四种时序路径



每条路径从时序路径**起点**开始，经过一些组合逻辑，然后在**终点**被捕获：

- Path 1: 从输入端口开始，到达时序元件的数据输入。
- Path 2: 从时序元件的时钟引脚开始，到时序元件的数据输入。
- Path 3: 从时序元件的时钟引脚开始，到输出端口结束。
- Path 4: 从输入端口开始，到输出端口结束。



布局 and 布线 (四)



- Placement:

运行脚本讲解 3

#Group Path setting

group_path -name reg2reg -from \$reg -to \$reg

group_path -name reg2cg -from \$reg -to \$ckgating

group_path -name in2reg -from \$input

group_path -name reg2out -to \$output

group_path -name feedthr -from \$input -to \$output

定义所有的路径类别

setPathGroupOptions reg2reg -effortLevel high

setPathGroupOptions reg2cg -effortLevel high

setPathGroupOptions in2reg -effortLevel low

setPathGroupOptions reg2out -effortLevel low

setPathGroupOptions feedthr -effortLevel low

setOptMode -ignorePathGroupsForHold \$ignore_path_groups 之前定义过

忽略inp2reg、reg2out、feedthr的hold冲突

./scripts/pr/placement.tcl

布局和布线（四）



- Placement:

运行脚本讲解 4

```
# -----  
# PlaceMode setting  
# -----
```

```
setPlaceMode -reset 重置placemode
```

```
setPlaceMode -place_global_ignore_scan true
```

```
setPlaceMode -place_global_reorder_scan false
```

```
setPlaceMode -place_global_place_io_pins false
```

```
setPlaceMode -place_detail_legalization_inst_gap 2
```

用于控制布局实例之间的间距。这个参数通常用于避免布局实例之间太过接近，以便进行后续的布线操作。

设置placemode的全局参数

忽略扫描链的布局，因为扫描链的位置通常是由自动布局工具决定

禁止重新排序扫描链。避免扫描链的位置重新排序，扫描链的位置通常是由自动布局工具决定

禁止在布局中放置I/O引脚。避免I/O引脚的位置受到布局的影响，因为I/O引脚的位置通常是由物理约束文件决定

`./scripts/pr/placement.tcl`

布局和布线（四）



- Placement:

运行脚本讲解 5

```
# ----- 设置全局路由层的范围。这个参数用于控制电路的布线，以便在保证时序和功耗的情况下，尽可能减少布线的层数。  
# global route layer setting  
# -----
```

```
setRouteMode -earlyGlobalMinRouteLayer $::env(MIN_GLOBAL_ROUTE_LAYER) -  
earlyGlobalMaxRouteLayer $::env(MAX_GLOBAL_ROUTE_LAYER)
```

```
setDesignMode -process $::env(PROCESS)  
设置设计模式，包括工艺过程、电压和温度等参数。
```

全局路由层的最小/大层数，由外界输入的两个环境变量控制

```
# -----  
# place the design & report congestion  
# -----
```

```
place_opt_design 执行布局placement操作  
reportCongestion -overflow
```

报告拥塞情况，包括拥塞的单元格数量和拥塞的比例。用于评估电路的布局质量。

./scripts/pr/placement.tcl

布局和布线 (四)



• Placement: 运行脚本讲解 6

设置时序分析报告的格式，包括
时序点、时序弧、时序网络、单
元格、负载、驱动器输出、上升
沿延迟、时钟延迟、到达时间和
总衰减等信息。

```
# -----  
# Output the time report 输出报告  
# -----  
  
set report_timing_format [list timing_point arc net cell fanout load slew incr_delay  
delay arrival total_derate ]  
setOptMode -timeDesignNumPaths 100 时序分析的路径数目  
set_table_style -no_frame_fix_width -nosplit 设置报告的表格样式，包括无边框、固  
timeDesign -preCTS \ 定宽度和不分割  
    -idealClock \  
    -pathReports \ 路径报告  
    -drvReports \ 驱动器报告  
    -slackReports \ 时间裕度报告  
    -numPaths 50 \ 生成的路径数  
    -prefix preCTS \ 生成报告的前缀，用于区分不同的报告  
    -outDir $::env(RESULT_DIR)/pr/report/placement_timing  
    指定输出报告的目录和文件名
```

这里的时序分析报告只是CTS之前的报告，
CTS之后会讲解怎样进行！

./scripts/pr/placement.tcl

布局 and 布线 (四)



- Placement:

运行脚本讲解 7

```
# -----  
# Save design  
# -----  
saveDesign $::env(RESULT_DIR)/pr/data/placement.enc  
exit
```

./scripts/pr/placement.tcl



布局 and 布线 (四)



- Placement:

报告讲解 1

```
#####  
# Generated by: Cadence Innovus 19.15-s075_1  
# OS: Linux x86_64(Host ID edaserver)  
# Generated on: Sun May 7 23:45:50 2023  
# Design: ibex_core  
# Command: timeDesign -preCTS -idealClock -pathReports -drvReports -slackReports -numPaths 50 -prefix preCTS -outDir ./result/pr/report/placement_timing  
#####
```

timeDesign Summary

Setup mode	all	reg2reg	in2reg	reg2out	default	feedthr
WNS (ns):	0.002	0.002	2.084	0.912	8.316	4.755
TNS (ns):	0.000	0.000	0.000	0.000	0.000	0.000
Violating Paths:	0	0	0	0	0	0
All Paths:	7863	4137	5649	101	1	32

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	0 (0)	0.000	1 (1)
max_tran	0 (0)	0.000	0 (0)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 40.411%
Routing Overflow: 1.32% H and 3.36% V

从整体可以看到这颗芯片的电路密度和布线溢出都较低，布局合理，且有很多空间溢出



./result/pr/report/placement_timing/preCTS.summary.gz

布局和布线（四）



- Placement:

1.Density（密度）：是指布局中使用的面积与可用面积之比。在布局过程中，需要尽可能利用可用的面积，同时避免出现过于密集的布局，以免影响布线和散热等因素。因此，Density通常用于评估布局的紧密程度和可用空间的利用率。在timedesign报告中，Density通常以百分比的形式呈现，越接近100%表示布局越紧密，越接近0%表示布局越松散。

2.Routing Overflow（布线溢出）：是指在布线过程中出现的信号线过长或过多，导致无法在规定的布线层数内完成布线。在数字IC设计中，布线溢出通常会导致电路的时序性能和功耗等方面的问题，因此需要进行优化和调整。在timedesign报告中，Routing Overflow通常以拥塞比例的形式呈现，越接近100%表示布线溢出越严重，需要进行优化和调整。



布局 and 布线 (四)



• Placement:

报告讲解 2

```
#####
# Generated by: Cadence Innovus 19.15-s075_1
# OS: Linux x86_64(Host ID edaserver)
# Generated on: Sun May 7 23:45:51 2023
# Design: ibex_core
# Command: timeDesign -preCTS -idealClock -pathReports -drvReports -slackReports -numPaths 50 -prefix preCTS -outDir ./result/pr/report/placement_timing
#####
Path 1: MET Setup Check with Pin gen_regfile_ff.register file i/rf_reg_q_reg[126]/CLK
Endpoint: gen_regfile_ff.register file i/rf_reg_q_reg[126]/D (v) checked with leading edge of 'core_clock'
Beginpoint: if_stage_i/instr_rdata_id_o_reg[22]/Q (^) triggered by leading edge of 'core_clock'
Path Groups: {reg2reg}
Analysis View: max_view
Other End Arrival Time 0.000
- Setup 0.351
+ Phase Shift 17.400
+ CPRP Adjustment 0.000
= Required Time 17.049
- Arrival Time 17.048
= Slack Time 0.002
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
Timing Path:
-----
Timing Point Arc Net Cell Fanout Load Slew Incr Delay Arrival Total
Delay Derate
-----
clk_i clk_i ^ - - - 0.000 - - 0.000 -
clk_i - - (net) 996 0.000 - - - 0.000 -
U5758/A - - sky130_fd_sc_hd_and2_1 - - 0.000 0.000 0.000 0.000 1.030
U5758/X A ^ -> X ^ - - sky130_fd_sc_hd_and2_1 - - 0.000 0.000 0.000 0.000 1.030
clk - - (net) 946 0.000 - - - - -
if_stage_i/instr_rdata_id_o_reg[22]/CLK - - sky130_fd_sc_hd_sedfxtp_1 - - 0.000 0.000 0.000 0.000 1.030
if_stage_i/instr_rdata_id_o_reg[22]/Q CLK ^ -> Q ^ - - sky130_fd_sc_hd_sedfxtp_1 - - 0.060 0.000 0.338 0.338 1.050
id_stage_i/imm_i_type[2] - - (net) 1 0.004 - - - - -
FE_OFC959_id_stage_i_imm_i_type_2/A - - sky130_fd_sc_hd_buf_4 - - 0.060 0.000 0.000 0.338 1.050
FE_OFC959_id_stage_i_imm_i_type_2/X A ^ -> X ^ - - sky130_fd_sc_hd_buf_4 - - 0.341 0.000 0.326 0.664 1.050
FE_OFN5614_id_stage_i_imm_i_type_2 - - (net) 10 0.121 - - - - -
FE_OFC960_id_stage_i_imm_i_type_2/A - - sky130_fd_sc_hd_inv_2 - - 0.342 0.000 0.022 0.686 1.050
FE_OFC960_id_stage_i_imm_i_type_2/Y A ^ -> Y v - - sky130_fd_sc_hd_inv_2 - - 0.107 0.000 0.139 0.826 1.050
```

reg2reg报告中分析了cts前时序路径的具体信息，这里我们可以查看哪条路径
时序比较紧张，在做cts时可以重点优化！

./result/pr/report/placement_timing/preCTS_reg2reg.tarpt.gz

布局和布线（四）



- 小结
 - PR data init: 输出PR需要的必要文件
 - Floor plan: 完成芯片面积、形状的确立，并明确IO口的摆放
 - Power plan: 对芯片加入电源网络(VDD/VSS)规划，并进行连接性和电源引脚短路检查
 - Placement: 将逻辑电路元件（如门、寄存器等）放置在芯片的物理位置上，并进行电路密度和布线溢出检查、对关键路径进行分析，为下一步CTS做准备

